Série BD : GROUP BY – HAVING – Jointure – Sous-requêtes – Date

Le directeur du restaurant « Dar Zitouna »

4ème SI – Lycée Rafèha – Enseignant : Aymen Essouyah

1. La situation

Le directeur du restaurant Dar Zitouna veut comprendre sa journée de travail. Chaque ticket de caisse est une ligne de la table `COMMANDE` ; chaque serveur est une ligne de la table `SERVEUR`.

Schéma : `SERVEUR (num, nom, salle, date_embauche)` – `COMMANDE (num, plat, categorie, prix, qte, numserveur, date_cmd)`

`prix` est le prix unitaire en DT. `numserveur` est une clé étrangère : elle référence `SERVEUR(num)`. `date_cmd` et `date_embauche` sont de type `DATE` (utilisées dans la partie D).

Table SERVEUR

num	nom	salle	date_embauche
1	Sami	Terrasse	2022-01-01
2	Rim	Salon	2023-09-01
3	Karim	Terrasse	2019-02-15
4	Nour	Salon	2024-01-10

Table COMMANDE

num	plat	categorie	prix	qte	numserveur	date_cmd
1	Salade tunisienne	Entrée	6	2	1	2026-03-15
2	Brik à l'oeuf	Entrée	4	5	2	2026-03-15
3	Couscous à l'agneau	Plat	22	3	1	2026-03-22
4	Poisson grillé	Plat	28	2	3	2026-03-28
5	Ojja merguez	Plat	18	4	2	2026-04-15
6	Makroudh	Dessert	5	6	3	2026-06-10
7	Assidat zgougou	Dessert	7	3	1	2026-11-15
8	Thé à la menthe	Boisson	3	8	2	2027-01-15
9	Jus d'orange	Boisson	4	5	3	2027-02-20
10	Brik au thon	Entrée	4	4	1	2027-03-15
11	Lablabi	Plat	9	2	2	2027-04-08
12	Eau minérale	Boisson	2	10	3	2027-04-14

2. Le chemin d'une requête

Imagine le directeur devant sa pile de tickets. Il traite toujours les clauses dans le même ordre, quel que soit l'ordre dans lequel tu les écris :



Les deux cases grisées (cadre épais) sont les deux filtres : `WHERE` trie les **tickets**, `HAVING` trie les **piles**.

N°	Clause	Le directeur se demande…	Ce qu'il fait
1	FROM	Quels tickets ai-je sur mon bureau ?	Il sort la table (ou les tables) concernée(s).
2	WHERE	Ce ticket m'intéresse-t-il ?	Il jette, un par un, les tickets qui ne conviennent pas. La jointure `table1.x = table2.x` s'écrit ici. Jamais de `COUNT`, `SUM`, `AVG`…
3	GROUP BY	Comment je range ce qui reste ?	Il fait une pile par valeur (une pile par catégorie, par serveur, par salle…).
4	HAVING	Cette pile m'intéresse-t-elle ?	Il garde ou jette des piles entières, d'après un résultat : `COUNT`, `SUM`, `AVG`, `MAX`, `MIN`.
5	SELECT	Que j'écris sur mon résumé ?	Une ligne par pile (ou par ticket s'il n'y a pas de `GROUP BY`).
6	ORDER BY	Dans quel ordre je présente ?	Il trie le résumé.

Lignes grisées : les deux filtres (`WHERE` et `HAVING`).

Règles à retenir

- `WHERE` parle d'un ticket (un champ, une comparaison). `HAVING` parle d'une pile (une fonction d'agrégat).
- Tout champ du `SELECT` qui n'est pas dans une fonction (`COUNT`, `SUM`…) doit aussi être dans le `GROUP BY`.
- Une jointure s'écrit dans le `WHERE` : `table1.x = table2.x`.
- On filtre d'abord les tickets (`WHERE`), puis on fait les piles (`GROUP BY`), puis on filtre les piles (`HAVING`).

Méthode en 4 questions

1. Que veut-on afficher, et une ligne pour quoi ? (la réponse après « pour chaque » donne le `GROUP BY`)
2. Quel calcul fait-on sur chaque pile ? (`COUNT`, `SUM`, `AVG`, `MAX`, `MIN`)
3. Y a-t-il une condition sur les tickets, avant de faire les piles ? (`WHERE`)
4. Y a-t-il une condition sur le résultat d'une pile ? (`HAVING`) – et faut-il une 2e table (jointure) ou une sous-requête ?

3. Exercices

Écris chaque requête dans le cadre prévu. Fais d'abord les piles « dans ta tête » avec les tableaux ci-dessus.

Partie A – Une seule table : COMMANDE

A1. Afficher, pour chaque catégorie de plat, le nombre de commandes.

Réponse :

A2. Afficher, pour chaque catégorie, la quantité totale commandée.

Réponse :

A3. Le chiffre d'affaires d'une commande est `prix × qte`. Afficher le chiffre d'affaires de chaque catégorie.

Réponse :

A4. Afficher les catégories dont le chiffre d'affaires dépasse 100 DT.

Réponse :

A5. Parmi les commandes dont le prix unitaire est supérieur ou égal à 5 DT, afficher le nombre de commandes de chaque catégorie.

Réponse :

A6. Parmi les commandes dont le prix unitaire est supérieur ou égal à 4 DT, afficher les catégories qui ont au moins 3 commandes, avec ce nombre.

Réponse :

A7. MariaDB refuse la requête suivante. Expliquer pourquoi, puis la corriger pour afficher les catégories ayant plus de 2 commandes.

```
SELECT categorie, COUNT(*)
FROM COMMANDE
WHERE COUNT(*) > 2
GROUP BY categorie;
```

Réponse :

Partie B – Deux tables : COMMANDE et SERVEUR (jointure)

B1. Afficher le nom de chaque serveur et le chiffre d'affaires des commandes qu'il a prises.

Réponse :

B2. Afficher les serveurs dont le chiffre d'affaires dépasse 120 DT.

Réponse :

B3. Afficher, pour chaque salle (Terrasse, Salon), le nombre de commandes prises par ses serveurs.

Réponse :

B4. Parmi les commandes de la salle Terrasse qui ne sont pas des boissons, afficher les serveurs qui ont pris au moins 3 commandes, avec ce nombre.

Réponse :

B5. Pour la requête de la question B4, complète le tableau : combien de lignes (ou de piles) reste-t-il à chaque étape ?

Étape	Nombre
Tickets de COMMANDE associés à leur serveur (jointure)	
Tickets restants après : `salle = 'Terrasse'` et `categorie <> 'Boisson'`	
Piles formées par `GROUP BY`	
Piles gardées par `HAVING COUNT(*) >= 3`	

Partie C – Sous-requêtes

C1. Afficher le serveur dont le chiffre d'affaires est le plus élevé.

Réponse :

C2. Afficher le nom des serveurs qui n'ont pris aucune commande.

Réponse :

C3. Afficher les plats (nom et prix) dont le prix unitaire est supérieur à la moyenne des prix de toutes les commandes.

Réponse :

C4. Afficher les catégories dont le chiffre d'affaires est supérieur à la moyenne des chiffres d'affaires des catégories.

Réponse :

Partie D – Fonctions sur les dates

Rappel des fonctions :

Fonction	Rôle
`DATE(chaine)`	extrait une date d'une chaîne au format DATE/DATETIME
`DAY(date)`, `MONTH(date)`, `YEAR(date)`	retournent le jour, le mois, l'année
`NOW()`	retourne la date et l'heure courantes
`ADDDATE(date, n)`	ajoute n jours (ou un intervalle, ex. `INTERVAL -5 YEAR`) à une date
`DATEDIFF(date1, date2)`	différence en jours entre deux dates
`TIMESTAMPDIFF(unité, d1, d2)`	différence entre deux dates dans l'unité donnée (`YEAR`, `MONTH`, `DAY`…)

D1. Afficher les commandes passées le 15 mars 2026.

Réponse :

D2. Afficher, pour chaque commande, le numéro, le plat et le jour du mois de la commande.

Réponse :

D3. Afficher les commandes passées au mois de mars 2026.

Réponse :

D4. Afficher, pour chaque année, le nombre de commandes.

Réponse :

D5. Afficher les commandes passées dans les 7 derniers jours par rapport à la date du jour (on suppose que la date du jour est le 15 avril 2027 ; les 7 derniers jours sont les 9, 10, …, 15 avril 2027).

Réponse :

D6. Afficher, pour chaque commande, le nombre de jours écoulés depuis la commande jusqu'au 31 décembre 2027.

Réponse :

D7. Afficher les serveurs embauchés depuis plus de 5 ans par rapport au 1er janvier 2027.

Réponse :

D8. Afficher, pour chaque serveur, son nom et le nombre de commandes qu'il a prises au mois de mars 2026.

Réponse :

D9. Afficher les catégories dont le chiffre d'affaires réalisé entre le 1er janvier 2027 et le 30 avril 2027 dépasse 50 DT.

Réponse :

D10. Expliquer pourquoi la requête suivante est incorrecte (elle est refusée dès que le mode `ONLY_FULL_GROUP_BY` est actif), puis la corriger.

```
SELECT num, plat, DAY(date_cmd)
FROM COMMANDE
WHERE DAY(date_cmd) = 15
GROUP BY YEAR(date_cmd);
```

Réponse :

D11. Afficher les commandes dont la date est comprise entre le 1er janvier 2026 et le 31 décembre 2026, triées par date croissante.

Réponse :

D12. Afficher, pour chaque serveur, la date de sa première commande et le nombre de jours entre cette première commande et le 31 décembre 2027.

Réponse :

D13. Afficher les commandes passées au cours du premier trimestre 2027 (janvier, février, mars).

Réponse :

D14. Afficher le chiffre d'affaires par mois pour l'année 2026.

Réponse :

D15. Afficher les serveurs dont la date d'embauche est postérieure au 1er janvier 2023, avec le nombre d'années écoulées jusqu'au 1er janvier 2027.

Réponse :